# APPENDIX A
# ARTIFACT APPENDIX

## A. Description & Requirements

This artifact demonstrates the paper's native-library analysis using the artifact evaluation test corpus. It collects ARM64 native libraries from 20 Android apps, matches them against a bundled Maven dataset, and produces summary tables. We request the *Available* and *Functional* badges.

The full study cannot be distributed because it contains confidential app data and millions of APK records. The artifact instead includes about 24 MB of pre-collected Maven data. It contains package metadata and native-library hashes, but no APKs, app records, or executable files. Collecting this data again requires substantial bandwidth and took more than two weeks, so the prepared dataset is included to keep evaluation practical.

*1) How to access:* The artifact is provided as `acid-artifact.zip` with the NDSS AE submission. A permanent archive link will be added to the camera-ready paper.

*2) Hardware dependencies:* Use a recent x86-64 Linux system with at least 1 GB RAM and 5 GB free disk space.

*3) Software dependencies:* Python 3.11 or newer, Internet access, and an AndroZoo API key are required. The scripts use only the Python standard library. Reviewers may obtain their own key at https://androzoo.uni.lu/access. We will also provide an evaluation key in the *Instructions* field on HotCRP. Please do not share this key with anyone else.

*4) Benchmarks:* The evaluator runs one collection experiment on 20 apps and one analysis experiment on the collected files. The exact apps and results may change as AndroZoo updates its catalog.

## B. Artifact Installation & Configuration

Extract the archive, enter its directory, and set the AndroZoo API key:

```
export ANDROZOO_API_KEY=<...>
```

Downloaded APK content is untrusted. Run the artifact as an unprivileged user and do not execute extracted native libraries.

## C. Experiment Workflow

Experiment E1 downloads only the APK portions needed to extract ARM64 native libraries. It stores the libraries and collection metadata in `artifact-output/`.

Experiment E2 verifies files, measures reuse, matches Maven binaries by `.text`/`.data`/`.rodata` SHA-256, and writes JSON and CSV reports.

## D. Major Claims

- (C1): Native libraries can be extracted from APKs using HTTP byte-range requests instead of routinely downloading complete APKs. E1 demonstrates this process.
- (C2): Native binaries can be grouped by file and ELF-section hashes, then matched against known Maven binaries. E2 demonstrates these analyses on the artifact evaluation test corpus.
- (C3): The paper's results over millions of apps, including coverage, library age, vulnerable versions, and manual security testing, are not reproduced because the required confidential data cannot be distributed.

## E. Evaluation

*1) Experiment (E1):* Required time: about 2 minutes of human effort and 15–45 minutes of computation and network time.

Run the bounded collection:

```
python3 new_dload_apks.py
```

The command should finish with `Collection complete`. It stops after collecting 20 apps with ARM64 native libraries. Completed apps are skipped if the command is run again. Runtime depends on AndroZoo and network speed, but should remain well within one day.

*2) Experiment (E2):* Required time: about 1 minute of human effort and less than 5 minutes of computation time.

Run the analysis:

```
python3 database_statistics.py --require-native
```

The command should finish with `Result: PASS`. E2 stores the generated CSV files in `artifact-output/`:

- `library_popularity.csv`: one row per popular binary, with its hash, size, app count, and example filename. It contains the artifact evaluation test corpus data corresponding to the paper's plot of the 500 most popular library hashes.
- `maven_source_summary.csv`: one row per package source, with ground-truth size and artifact evaluation test corpus match counts. It corresponds to the paper's source pie chart and source-usage table in RQ1.
- `maven_matches.csv`: the main result. Each row maps an app binary to a detected library name, developer or package identifier, and candidate version. Most importantly, it reports version information. The paper's downstream CVE analysis uses these mappings to count apps that may contain an affected version. This file also relates to the paper's table of popular Maven packages. One binary may occur in several packages or versions, so rows are candidate mappings rather than unique attributions.

These files describe only the artifact evaluation test corpus. The related paper figures and tables use the full corpus.

## F. Customization

Evaluators may provide their own APK list with `--input apps.csv`. Run either script with `--help` for available settings.